

**ÇANKAYA UNIVERSITY
COMPUTER ENGINEERING
DEPARTMENT**

CENG 407

Discussion

TARAS: Agricultural Analysis System

ARDA KAHRAMAN - 202111002
HATİCE ELİF ARABACI - 202211008
ÖYKÜ EYLÜL BOZDAĞ - 202311410
MUSTAFA BERK KESKİN - 202211042

TABLE OF CONTENTS

1. Introduction.....	2
1.1. Purpose of the Document.....	2
1.2. AI/ML Scope of the Project.....	3
2. Disease Detection Module.....	3
2.1. Problem Definition.....	3
2.2. Dataset and Data Preparation.....	4
2.3. Model Selection.....	5
2.4. Training Strategy.....	6
2.5. Experimental Results.....	7
2.6. Interpretation of Results.....	8
2.7. Comparison with Related Studies.....	9
2.8. Limitations of the Module.....	10
3. LLM Module.....	11
3.1. Problem Definition.....	11
3.2. System Design.....	11
3.3. Model Selection and Explanation.....	12
3.4. Benefits and Challenges.....	13
3.5. Comparison with Related Studies.....	13
3.6. Limitations of the Module.....	14
4. Future Work.....	15
4.1. Disease Detection Improvements.....	15
4.2. LLM Module Improvements.....	15
5. Conclusion.....	16
6. References.....	16

1. Introduction

1.1. Purpose of the Document

This chapter discusses the AI and ML components of the TARAS precision-agriculture platform. Rather than repeating implementation details from the System Architecture and Methodology report, it interprets and justifies the design decisions behind the two ML-related modules, reflects on their results, and compares them with relevant literature. It explains why the selected approaches fit the problem, what the experimental evidence shows, and where the current design has limitations.

The Disease Detection and LLM-based Advisory modules are treated as separate subsystems. Although both support the same decision-support workflow, they solve different problems: one is visual and discriminative, while the other is linguistic and generative. Therefore, they are evaluated with different criteria and compared against different related work.

When result tables, performance metrics, or architectural diagrams are mentioned, the relevant sections of the main report are referenced. If the current system differs from the originally documented version, those differences are stated explicitly.

1.2. AI/ML Scope of the Project

TARAS includes several data-driven subsystems, but only two are machine-learning components in the strict sense: the vision-based plant disease detection module and the LLM-based advisory module. The disease detection module classifies leaf images using an ensemble of CNN and transformer backbones (ConvNeXt and Swin), while the advisory module interprets sensor readings, irrigation outputs, disease results, and user input to generate natural-language guidance.

The irrigation recommendation subsystem is excluded from this chapter. Although it is central to TARAS, it is based on calibrated agronomic calculations using reference evapotranspiration (ET_0), crop coefficient (K_c), and field sensor observations, following the FAO-56 methodology. Since this pipeline is deterministic and rule-based rather than learned from data, it is not evaluated here using classification metrics or learning-based arguments.

These two ML modules serve different purposes and are therefore discussed separately. Disease detection is a discriminative image-classification task, while the LLM advisory module is a generative, context-aware task that combines structured system outputs with user text. Because they differ in input type, output form, and evaluation criteria, they are compared against different bodies of related work.

2. Disease Detection Module

2.1. Problem Definition

The disease detection module identifies plant diseases from a single RGB leaf photo and returns one disease-or-health label with a confidence score. The current model covers fourteen treatment-distinct, cross-crop-merged classes for bell pepper, corn, potato, squash, and tomato workflows, with additional healthy cherry and peach samples used during training. The deployed labels include `bacterial_spot`, `early_blight`, `late_blight`, `healthy`, `powdery_mildew`, `target_spot`, `yellow_leaf_curl_virus`, and other crop-specific disease classes.

This module is important because early diagnosis can prevent serious yield loss, especially for fast-spreading diseases such as late blight, bacterial spot, early blight, and leaf mold. It is designed to help small-scale farmers who may not have regular access to an agronomist.

The main challenge is that field images vary in lighting, angle, occlusion, and background complexity. Some diseases also look visually similar in early stages, so the model must distinguish subtle cues such as lesion shape, margins, patterns, and small structures.

Within TARAS, the module works both as a standalone diagnostic tool and as an input to the LLM-based advisory module, which combines disease results with humidity, irrigation, and field history to generate contextual guidance.

2.2. Dataset and Data Preparation

The disease detection module uses a hybrid dataset combining clean laboratory images and real-world field images. PlantVillage provides a large, controlled lab dataset with healthy and diseased leaves on plain backgrounds, but it does not fully represent field conditions.

To reduce this gap, TARAS also uses real-world sources: a curated RW dataset collected from permissively licensed sources such as iNaturalist, Wikimedia Commons, extension services, and open repositories, plus PlantDoc, which includes natural backgrounds, varied lighting, multiple leaves, and field-like conditions. Each RW image is tracked with metadata such as source URL, licence, label confidence, and visual difficulty.

The datasets are combined to address domain shift. PlantVillage is large but less realistic, while RW and PlantDoc images are more realistic but smaller and noisier. The hybrid dataset benefits from both scale and field variability.

The real-world blend is small, about 40–74 images, and focuses on poorly covered classes, especially `spider_mites`. Images are reviewed for

symptom clarity, background complexity, leaf visibility, and label reliability. Scarce classes are prioritized, and duplicates or near-duplicates are removed using perceptual hashing.

All sources use the same preprocessing pipeline: format and resolution checks, resizing, color standardization, and backbone-specific normalization. Augmentation, including flips, small rotations, brightness/contrast changes, and mild crop or zoom, is applied only to the training split. Validation and test images are not augmented. The held-out test set is tagged by source, with PlantDoc used as the field benchmark and PlantVillage as the clean reference.

The final training dataset used for the current production candidate contains 28,600 training images across 14 classes. The held-out test set (n=1,941) is composed by source as follows:

Class	Field (PD)	Lab (PV)	RW	Total
bacterial_spot	39	110	0	149
corn_common_rust	37	113	0	150
corn_gray_leaf_spot	18	77	0	95
corn_northern_leaf_blight	50	100	0	150
early_blight	22	126	0	148
healthy	21	129	0	150
late_blight	40	107	0	147
leaf_mold	16	133	0	149
mosaic_virus	0	56	0	56
powdery_mildew	37	113	0	150
septoria_leaf_spot	36	113	0	149
spider_mites	0	133	15	148
target_spot	0	150	0	150
yellow_leaf_curl_virus	16	134	0	150
Total	332	1594	15	1941

A validation set of 5,383 images is used for checkpoint selection. During model selection, TARAS maximizes a composite score of 0.5 x validation macro F1 plus 0.5 x field (PlantDoc) macro F1, so that real-world behaviour and clean-data stability are weighted equally.

A single held-out test set of 1,941 images is maintained and tagged by image source. Its field slice (PlantDoc photographs, 332 images) is the principal real-world evaluation used for deployment decisions, while its lab slice (PlantVillage, 1,594 images) provides

a clean, controlled reference; a small real-world blend (15 images) is also included.

2.3. Model Selection

The final deployment configuration is a three-member softmax-averaging ensemble, ConvNeXt-V2-Base, Swin-Small (384×384, window 12), and a field-oversampled Swin-Base, selected by a brute-force search over all three-member subsets of a 14-model candidate pool, ranked by a validation composite of 0.5·macro-F1 + 0.5·field-F1 (analogous to Caruana et al.'s ensemble-selection method [4]). The winning equal-weight trio is a strict Pareto improvement over the earlier five-member ensemble: equal or better on every test metric while being ~40% faster and ~38% smaller. It reaches 0.8488 field macro-F1 and 0.9697 full-test macro-F1, with temperature calibration cutting ECE from ~10% to ~1.4%. Single-backbone benchmarking (in which Swin-Small was the strongest individual model) is what justified building the ensemble; no single model is deployed on its own. Each member contributes orthogonal signal, ConvNeXt-V2's FCMAE pretraining[2], Swin-S's high-resolution input, and Swin-B's field-biased oversampling. These results suggest that the ensemble generalizes well across both real-world and balanced evaluation conditions.

Swin-Small is also suitable for the task from a modelling perspective. After label merging, the main challenge is less about broad class separation and more about fine-grained visual differences between similar diseases. Windowed self-attention allows the model to capture local lesion details while still considering broader leaf structure, which is useful for difficult pairs such as bacterial_spot vs. septoria_leaf_spot and early_blight vs. late_blight.

For deployment, the trained PyTorch model is converted to ONNX and served on AWS Lambda using ONNX Runtime. This replaced the earlier TensorFlow/Keras and TFLite path because it supports smaller container images, simpler CPU-only inference, and a clearer separation between training and serving. Before promotion, PyTorch and ONNX outputs are checked on fixture images, and a parity gate rejects any export whose maximum absolute softmax difference exceeds 0.005.

2.4. Training Strategy

The current system uses a unified, single-stage training strategy instead of the earlier curriculum-based approach. This choice makes the pipeline simpler, more stable, and easier to reproduce while still allowing the model to learn from both laboratory and real-world images.

The final training set combines PlantVillage (PV) images with the curated real-world (RW) dataset. This exposes the model to both clean, label-consistent laboratory images and more variable field-style images during the same training process. Instead of applying a staged transition from PV to RW data, domain

balance is handled through sampling and loss weighting.

Class and domain imbalance are handled on the loss side rather than by resampling. Lab (PlantVillage) samples are down-weighted in the loss (composite weight 0.5) so the scarcer field samples contribute more gradient, and real-world sources are up-weighted (real-world x5, and PlantDoc x8 in the field-oversampled run). Per-class weighting was tested but it consistently hurt the hardest class, so it is not used in the final model.

Data augmentation is applied only to the training split. The pipeline includes random resized cropping, horizontal flipping, rotation, and mild brightness and contrast changes. These augmentations improve robustness to differences in viewpoint, lighting, and background while preserving the disease-specific visual patterns needed for classification.

The evaluation protocol is explicitly designed to reflect deployment conditions. Two separate test sets are maintained:

- **field slice: PlantDoc photographs only, representing the primary benchmark for deployment performance.**
- **lab slice: clean PlantVillage images, used to analyse per-class behaviour under a controlled distribution.** Model selection is based on a composite score that weights the full validation set and the real-world field (PlantDoc) slice equally:
- **the full validation set macro F1, weighted 0.5 • the real-world field (PlantDoc) macro F1, weighted 0.5**

This weighting ensures that the selected model prioritises real-world generalisation while still retaining sensitivity to clean, canonical visual patterns.

2.5. Experimental Results

The most important numerical result of this work is the distinction between the final deployment candidate and the broader comparison set. The deployed three-member ensemble achieves 0.9701 accuracy and 0.9697 macro F1 on the full held-out test set, and 0.8488 macro F1 on the real-world field slice (with 0.9944 macro F1 on the clean lab slice).

Metric	Value
Test Macro F1 (overall)	0.9697
Test Accuracy	0.9701
Field Macro F1 (PlantDoc n=332)	0.8488
Lab Macro F1 (PlantVillage n=1594)	0.9944
Validation composite (selection)	0.9273
Minimum per-class field F1	0.6486

The single-backbone benchmark on the revised dataset is what motivated building the ensemble: no individual backbone matched the three-member ensemble's balance of field and overall macro F1, so the deployed system averages the calibrated softmax of ConvNeXt-V2-Base, Swin-Small (384), and Swin-Base rather than shipping any single backbone.

Backbone	Params	Input	Val composite	Test Macro F1	Test Field F1	Role / verdict
ConvNeXt-V2-Base (FCMAE)	88M	224	0.8474	0.9594	0.8042	Top ensemble contributor
ConvNeXt-V1-Base	88M	224	0.8651	0.9678	0.8290	Strong single backbone
Swin-Base @384	87M	384	0.8701	0.9696	0.8332	Distant-leaf robust
Swin-Base @224	88M	224	0.8747	0.9665	0.8204	Core member
Swin-Small @224	49M	224	0.8662	0.9649	0.8224	Efficient peer
Swin-Small @384	49M	384	—	—	0.8348	Ensemble member (highest single field F1)
Swin-Tiny @224	28M	224	0.8433	0.9648	0.8204	Redundant with Swin-S/B
Swin-Base + class weights	88M	224	0.8377	0.9637	0.8053	Class weights hurt hardest class
CAFormer-S36	37M	224	0.8704	0.9623	0.8207	Screened; no test gain
Hiera-Base	51M	224	0.831	0.9605	0.7776	Screened
MaxViT-Small	68M	224	—	—	—	Training failed (NaN on ROCm)

Per-class results of the deployed ensemble on the real-world field slice show:

Class	Field-test n	Field F1
powdery_mildew	37	1
corn_common_rust	37	0.986
leaf_mold	16	0.968
late_blight	40	0.964
yellow_leaf_curl_virus	16	0.933
healthy	21	0.909
corn_northern_leaf_blight	50	0.86
bacterial_spot	39	0.719

septoria_leaf_spot	36	0.694
early_blight	22	0.655
corn_gray_leaf_spot	18	0.649
Field Macro F1 (11 classes)	332	0.849

2.6. Interpretation of Results

The revised results support a balanced conclusion: no single architecture is inherently superior. Final performance depends strongly on label design, dataset quality, and model selection.

Candidate models were trained under the same setup, and the final classifier was selected based on measured performance rather than architectural assumptions. The real-world field slice was treated as the main deployment-readiness benchmark, while the lab-heavy full test set served as a secondary sanity check.

The deployed ensemble achieves 0.8488 macro F1 on the real-world field slice, 0.9697 macro F1 on the full test set, and 0.9944 on the clean lab slice. This performance gap reflects the difficulty of uncontrolled field imagery rather than weakness on clean data.

Remaining errors mainly occur between visually similar leaf-spot and blight classes, such as bacterial_spot vs. septoria_leaf_spot or early_blight vs. late_blight. This shows the limits of single-image diagnosis without extra crop metadata or multiple images.

Overall, the final TARAS classifier is a deployment-oriented model selected after dataset revision, label-schema redesign, model comparison, and error analysis, not only a laboratory benchmark model.

2.7. Comparison with Related Studies

The classic reference point remains Mohanty, Hughes and Salathe (2016), who trained deep CNNs on 54,306 PlantVillage images and reported 99.35% accuracy on a held-out test set^[1]. That result established the feasibility of deep learning for plant disease detection, but it also exemplifies the main limitation that later studies exposed: excellent laboratory accuracy does not guarantee strong field performance^[5]. TARAS deliberately accepts lower headline numbers in exchange for evaluation on more realistic imagery and therefore addresses a different question than PlantVillage-only studies.

A useful field-oriented comparison is PlantDoc and later work built around it. Introduced PlantDoc as a challenging real-world benchmark with cluttered backgrounds and cross-domain variability ^[6], and later object-detection

and classification studies continued to use it precisely because it is harder than PlantVillage. Relative to this line of work, TARAS is similar in spirit because it prioritises natural images, but it differs in being a task-specific hybrid corpus with label redesign guided by error analysis rather than a fixed public benchmark alone.

For comparison with modern transformer-based approaches, recent studies based on Swin Transformer architecture have demonstrated strong performance under natural and noisy image conditions[3], [7]. TARAS is aligned with these findings: the final selection of a transformer-based ensemble over the revised 14-class dataset supports the claim that transformer-style feature modelling can be advantageous once the label schema and data preparation are appropriate.

The main point of difference is methodological. Most published papers compare architectures on a fixed benchmark, whereas TARAS also redesigns the label taxonomy when the benchmarked error patterns show that a finer distinction is not reliable enough for deployment. This is why the final document reports not only architecture comparison but also the decision to merge classes by treatment across crops, for example collapsing early blight and late blight on both tomato and potato into single, crop-agnostic `early_blight` and `late_blight` classes. For a live agricultural system, this decision is more meaningful than preserving a more granular taxonomy that produces unstable advice.

Taken together, the literature suggests that TARAS occupies an intermediate position. It does not chase PlantVillage-only accuracy numbers, and it does not depend exclusively on one public field dataset either. Instead, it uses a curated hybrid dataset, a deployment-oriented test split and a model-selection process that is directly tied to the target application

2.8. Limitations of the Module

Several limitations follow from the current design and should be acknowledged clearly. First, the classifier is limited to fourteen categories across five crops. Images outside this scope, such as different crops, unseen diseases, nutrient deficiencies, pest damage, or abiotic stress, may still be forced into one of the known classes. Since the model does not currently include an out-of-distribution rejection class, the backend confidence threshold is the only mechanism for reducing unreliable predictions, and this is not a perfect solution.

Second, the real-world dataset is still relatively small. Although source diversity was considered during collection, the dataset reflects the regions, seasons, and crop conditions available in permissively licensed image sources. As a result, the model is likely to perform better in contexts similar to the training data and less reliably in underrepresented conditions, such as specific greenhouse cases in Türkiye. Expanding the real-world dataset remains an important future improvement.

Third, some classes remain weaker because of dataset imbalance. PlantVillage contains fewer examples for classes such as mosaic_virus and healthy, and augmentation cannot fully replace real image diversity. The curated real-world data helps reduce this issue, but these classes still represent weak points in the current model.

Fourth, the module works at the image level. It classifies one leaf image at a time and does not reason across multiple images of the same plant or track symptom progression over time. This limits performance in early-stage disease cases, where a single leaf may be ambiguous but several images over time could make the diagnosis clearer.

Finally, the module is only a visual diagnostic aid. Some plant-health problems, such as over-irrigation, nutrient deficiency, or root disease, may produce visible symptoms without being identifiable from the image alone. The LLM-based advisory module partially addresses this by combining disease predictions with sensor and irrigation history, but the image classifier should not be treated as a standalone diagnostic authority. Its role is to narrow down possible causes and support further investigation.

3. LLM Module

3.1. Problem Definition

The LLM-based advisory module addresses a different but complementary problem to disease detection: it turns system outputs into explanations that users can understand and act on. Soil moisture values, irrigation recommendations, and disease confidence scores are useful only if the user can interpret what they mean for their field. Since TARAS is designed for non-expert users, this translation layer is an important part of the platform.

The module answers questions such as “What do these values mean for my field today?” and “What should I do next?” using the context of the user’s farm, field, zone, and crop. Its responses are generated from structured inputs such as live sensor readings, recent irrigation decisions, disease detection results, and user-provided information. The goal is not to create an open-ended chatbot, but a decision-support assistant whose answers remain grounded in the data available in the system.

The main design challenge is balancing usefulness and reliability. The module needs to be flexible enough to answer different user questions, but constrained enough to avoid common LLM problems such as hallucinated details, overconfident statements, or plausible but incorrect recommendations. Therefore, the advisory system is designed to produce practical guidance while keeping its reasoning tied to concrete data sources and curated agricultural knowledge.

3.2. System Design

The advisory module is implemented as an agentic pipeline instead of a single-shot prompt. When the user asks a question, the model does not answer only from its general training knowledge. It first decides whether it needs to call one or more read-only tools to ground the answer. These tools can retrieve live database records, search the curated agricultural knowledge base, or direct the mobile app to a relevant screen when visual information is more useful than text. The final response is generated only after the tool results are returned.

The agent currently has access to eleven tools: one navigation tool, nine live-data read tools (field overview, latest zone reading, zone history, zone configuration, irrigation history, sensor diagnostics, carbon summary, disease history, and active alerts), and one knowledge-base search tool. To keep the system controlled, the tool-use loop is capped at eight iterations with a thirty-second wall-clock timeout, plus per-tool input schemas, user-scoped data access, and capped result sizes.

Domain grounding is mainly provided through the `search_knowledge` tool. This tool uses PostgreSQL full-text search over a curated knowledge base of about 146 fragments covering crop guides, diseases, pests, soils, fertilizers, irrigation practices, climate considerations, and system usage. Instead of relying only on the LLM's internal knowledge for agronomic advice, the system retrieves relevant project-controlled guidance and lets the model reason over it. This also allows the advisory knowledge to be updated without retraining the model.

The second grounding mechanism is the system prompt. It is compact, cached, and designed to enforce a few key behaviours: answer in the user's language, stay short and direct unless more detail is needed, use tools for user-specific data, and use navigation only when a visual screen would genuinely help. This text-first design was chosen because earlier versions overused navigation and interrupted the user's flow.

3.3. Model Selection and Explanation

The production model is Anthropic Claude Haiku 4.5 (claude-haiku-4-5-20251001), accessed through the Anthropic TypeScript SDK. Haiku 4.5 was chosen because, as a frontier model, it gave the most reliable agentic behaviour: native structured tool calls, consistent adherence to the brevity and language rules, and none of the failure modes such as mixed-script output or malformed tool calls seen in the open-weight candidates, while remaining fast and inexpensive enough for interactive use. The system also includes a fallback path that runs the same agentic tool loop through Groq, using openai/gpt-oss-120b by default (Qwen3 was evaluated and rejected because it produced mixed-script Turkish output). The fallback uses a separately tuned, structurally simpler system prompt adapted to open-weight models, but it

exposes the same eleven tools and the same eight-iteration loop.

The agentic design was selected over simpler alternatives such as a single-shot prompt or fixed retrieval pipeline. Different user questions require different parts of the system state: soil-moisture questions may need zone history, disease-related questions may need irrigation history, and explanation questions may need the knowledge base. Instead of inlining all possible context into every request, the tool-loop lets the model retrieve only what is relevant. This keeps responses cheaper, more focused, and easier to trace through the selected tool calls.

3.4. Benefits and Challenges

The main benefit of the advisory module is accessibility. Raw values such as soil moisture, crop coefficient, or disease confidence scores are difficult for non-expert users to interpret on their own. The advisory module turns these outputs into practical explanations, such as why a specific zone needs irrigation today while another does not. In this sense, it makes the platform's technical outputs usable for the intended audience.

The module also acts as an integration layer across TARAS subsystems. Disease results, irrigation recommendations, sensor readings, and field history are stored separately, but user questions often combine them. For example, a question such as "Is this leaf spot related to overwatering?" requires disease output, irrigation history, and humidity data together. The tool-loop design allows the module to retrieve and connect these pieces without requiring the user to understand the underlying database structure.

Another benefit is explainability. Because responses are grounded in tool outputs and retrieved knowledge, the module can refer to the specific factors behind an answer, such as current soil moisture, growth stage, irrigation history, or a relevant disease guide. This makes the response more trustworthy than a generic chatbot answer.

The main challenges are accuracy, latency, cost, reliability, and trust boundaries. The grounding mechanisms reduce the risk of hallucination, but they do not eliminate it, especially for edge cases outside the curated knowledge base. The agentic tool loop also adds latency because some questions require multiple tool calls, although streaming improves perceived response time. Cost is controlled through rate limits and result-size caps, while provider outages are handled through the Anthropic/Groq fallback setup. Finally, the advisory output must be framed as decision support, not as a replacement for expert diagnosis, especially for high-risk cases such as possible late blight.

3.5. Comparison with Related Studies

Recent studies on LLMs in agriculture, such as Wang et al. (2024), identify the

same problem addressed by the TARAS advisory module: smart-farming systems can produce useful analytical outputs, but farmers may struggle to interpret them. LLMs can help by translating sensor readings, model outputs, and agronomic recommendations into natural-language explanations. TARAS follows the retrieval-augmented generation (RAG) approach rather than fine-tuning: Claude Haiku 4.5 is grounded with a curated agricultural knowledge base retrieved through PostgreSQL full-text search.

Most agricultural chatbot systems fall into two broad groups. Rule-based systems are predictable and explainable, but they are limited to predefined questions. Free-form LLM chatbots are more flexible, but they can produce unsupported or incorrect advice. TARAS is designed to sit between these two approaches. Users can ask questions naturally, but the model is constrained by tool use, scoped live data access, and retrieval grounding.

The TARAS advisory module differs from many agricultural LLM systems in three main ways. First, it connects to live sensor, irrigation, disease, diagnostic, and carbon data through scoped and audit-logged tools. Second, it remains text-first and uses navigation only when a screen view would help the user more than a written answer. Third, it includes a provider fallback path, so the advisory function can continue in a simpler form if the primary LLM provider is unavailable.

3.6. Limitations of the Module

The advisory module depends heavily on the quality of its upstream data. If a zone has faulty sensors, stale readings, or inconsistent metadata, the generated response may also be unreliable, even if it sounds fluent. The `get_sensor_diagnostics` tool helps reduce this risk by allowing the agent to check sensor-health information and add caveats when needed, but the module cannot be more accurate than the data it receives.

Retrieval quality is another limitation. The curated knowledge base contains about 146 fragments across eight categories, so it is useful but not encyclopedic. Questions that match its coverage can be grounded well, while questions outside its scope may rely more on the model's general knowledge. For this reason, expanding and maintaining the knowledge base remains an important future task.

The module should also not be treated as a replacement for expert agronomic judgment. In complex or high-risk cases, such as unusual disease symptoms, multiple stress factors, or decisions with serious economic consequences, the advisory response should be used as a starting point rather than a final authority.

The agentic tool loop introduces some tool-specific risks, such as

selecting the wrong tool, misreading a tool result, or answering with incomplete context. These risks are reduced through iteration limits, result-size caps, scope checks, and audit logging, but they cannot be removed completely.

Finally, the module requires an internet connection. This may be a limitation in rural areas with unstable connectivity. At present, the mobile client can show the last-known advisory state, but it cannot generate new advisory responses offline. An offline-capable lightweight model could be considered as future work.

4. Future Work

Both ML modules are at a point where the core design is stable and the remaining work is largely incremental, extending coverage, improving robustness, and closing specific gaps identified during development. The items below are grouped by module and ordered loosely by estimated impact on the user experience.

4.1. Disease Detection Improvements

Future improvements for the disease-detection module should build on the revised 14-class ensemble system rather than returning to a broader but less reliable label set. The first priority is targeted dataset expansion for the hardest categories, especially the `bacterial_spot/septoria_leaf_spot` and `corn leaf-spot` pairs. Adding more real-world images from Turkish greenhouse and open-field conditions would likely improve performance more than switching to a new backbone.

A second priority is confidence calibration and rejection handling. Although the current model gives strong top-1 predictions, production use would benefit from calibrated thresholds and a fallback path for uncertain cases. For example, the system could return “low confidence, please capture another image or use the advisory module” instead of always forcing a class.

Future architecture experiments should also be selective. The current results already identify Swin-Small as a strong deployment candidate, with credible alternatives such as ConvNeXt-Small. Therefore, the next step should focus on improving the deployed ensemble pipeline and occasionally testing it against a small number of strong baselines.

4.2. LLM Module Improvements

The LLM advisory module has four main future improvements.

First, the knowledge base should be expanded. The current 146 curated fragments cover common cases well, but unusual pest–disease interactions, region-specific crop guidance, and less common soil types need more coverage. New content should remain curated and attributable rather than scraped from the

open web.

Second, the tool set can be extended. Useful additions include a weather-forecast tool, a historical-analogue tool for comparing current conditions with similar past cases, and a push-notification tool for proactive alerts.

Third, offline and low-bandwidth support should be considered. A small on-device fallback model could answer a narrow set of common questions using cached field data, such as irrigation status, recent disease results, and basic crop-stage guidance.

Fourth, the module needs a feedback loop. A simple thumbs-up/thumbs-down mechanism would help identify weak answers, improve the system prompt, and guide future knowledge-base updates.

Additional improvements could include conversation summarisation, voice input, what-if irrigation scenarios, and more detailed per-tool authorization.

5. Conclusion

This chapter discussed the two AI/ML components of TARAS: the vision-based disease detection module and the LLM-based advisory module. Although they solve different problems, both support the same goal: helping users interpret field data and make better agricultural decisions.

For disease detection, the project converged on a revised 14-class label schema and, after benchmarking individual backbones, selected a three-member ensemble (ConvNeXt-V2-Base, Swin-Small at 384, and Swin-Base) as the final deployment configuration. On the real-world field test slice the ensemble reaches 0.8488 macro F1, and 0.9697 macro F1 on the full balanced test set. These results show that the strongest improvements came not only from model architecture, but from aligning the label schema, dataset, and evaluation protocol with real deployment conditions.

The LLM advisory module addresses a different need: turning structured system outputs into understandable guidance. Its agentic tool-loop design, scoped read-only tools, curated knowledge base, and provider fallback were chosen to balance usefulness with reliability. The module has been demonstrated end-to-end in production, with typical response times of three to six seconds, but it has not yet been evaluated through a large-scale quantitative study. Its current evaluation is therefore mainly qualitative, based on coherence, grounding, and consistency with system outputs.

The current limitations are clear: the classifier is limited to fourteen classes, the advisory knowledge base is compact, the LLM module depends on cloud connectivity, and the classifier does not yet provide visual explanation maps. However, these are incremental improvement areas rather than reasons to redesign the system from scratch.

Overall, the AI/ML work in TARAS follows a consistent design philosophy: both modules should be useful without overclaiming, flexible without becoming ungrounded, and supportive without replacing expert judgment. The disease classifier and advisory agent are therefore best understood as decision-support tools that help non-expert users make better use of the data produced by their own fields.

6. References

- [1] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using Deep Learning for Image-Based Plant Disease Detection," *Frontiers in Plant Science*, vol. 7, p. 1419, 2016.
- [2] S. Woo, S. Debnath, R. Hu, X. Chen, Z. Liu, I. S. Kweon, and S. Xie, "ConvNeXt V2: Co-designing and Scaling ConvNets with Masked Autoencoders," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, Vancouver, BC, Canada, 2023, pp. 16133–16142.
- [3] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo, "Swin Transformer: Hierarchical Vision Transformer using Shifted Windows," in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, Montreal, QC, Canada, 2021, pp. 10012–10022, doi: 10.1109/ICCV48922.2021.00986.
- [4] R. Caruana, A. Niculescu-Mizil, G. Crew, and A. Ksikes, "Ensemble selection from libraries of models," in *Proc. 21st International Conference on Machine Learning (ICML)*, Banff, AB, Canada, 2004, Art. no. 18, doi: 10.1145/1015330.1015432.
- [5] J. G. A. Barbedo, "Impact of dataset size and variety on the effectiveness of deep learning and transfer learning for plant disease classification," *Computers and Electronics in Agriculture*, vol. 153, pp. 46–53, 2018, doi: 10.1016/j.compag.2018.08.013.
- [6] D. Singh, N. Jain, P. Jain, P. Kayal, S. Kumawat, and N. Batra, "PlantDoc: A dataset for visual plant disease detection," in *Proc. 7th ACM IKDD CoDS and 25th COMAD*, Hyderabad, India, 2020, pp. 249–253, doi: 10.1145/3371158.3371196.
- [7] Y. Guo, Y. Lan, and X. Chen, "CST: Convolutional Swin Transformer for detecting the degree and types of plant diseases," *Computers and Electronics in Agriculture*, vol. 202, Art. no. 107407, 2022, doi: 10.1016/j.compag.2022.107407.